1. General settings

n = 64
n_train = 8192
batch_size = 32
n_epochs = 200
FNO width = 32
Retained FNO modes = 16
Number of FNO blocks = 3
Decaying Adam learning rate = 0.0009
Lambda_f = 20.0
Phase_contrast = 5.0
Reference solver is the previously studied FFT-based solver with no Fourier-mode truncation (all modes are used).

ZSSR is always tested at n = 256

2. Model architectures

2.1 Regular FNO:

The model simply has 3 FNO blocks with each block sharing the exact same structures (spectral branch + bypass branch with GELU at the end). The code for each FNO block is as follows:

```python
class FNOBlock2d(eqx.Module):
    spectral_conv: SpectralConv2d
    bypass_conv: eqx.nn.Conv2d


    def __init__(self, width, modes, *, key):
        k1, k2 = jax.random.split(key)
        self.spectral_conv = SpectralConv2d(width, modes, key=k1)
        self.bypass_conv = eqx.nn.Conv2d(width, width, kernel_size=1, key=k2)


    def __call__(self, x):
        # Add spectral and bypass branches together, apply GELU
        return jax.nn.gelu( self.spectral_conv(x) + self.bypass_conv(x) )
```

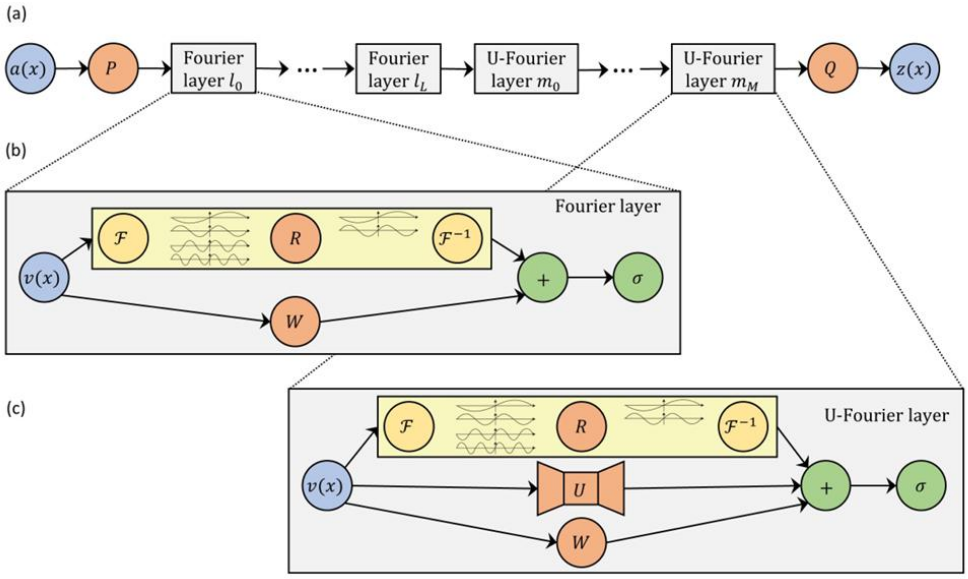
Python

2.2 U-FNO:

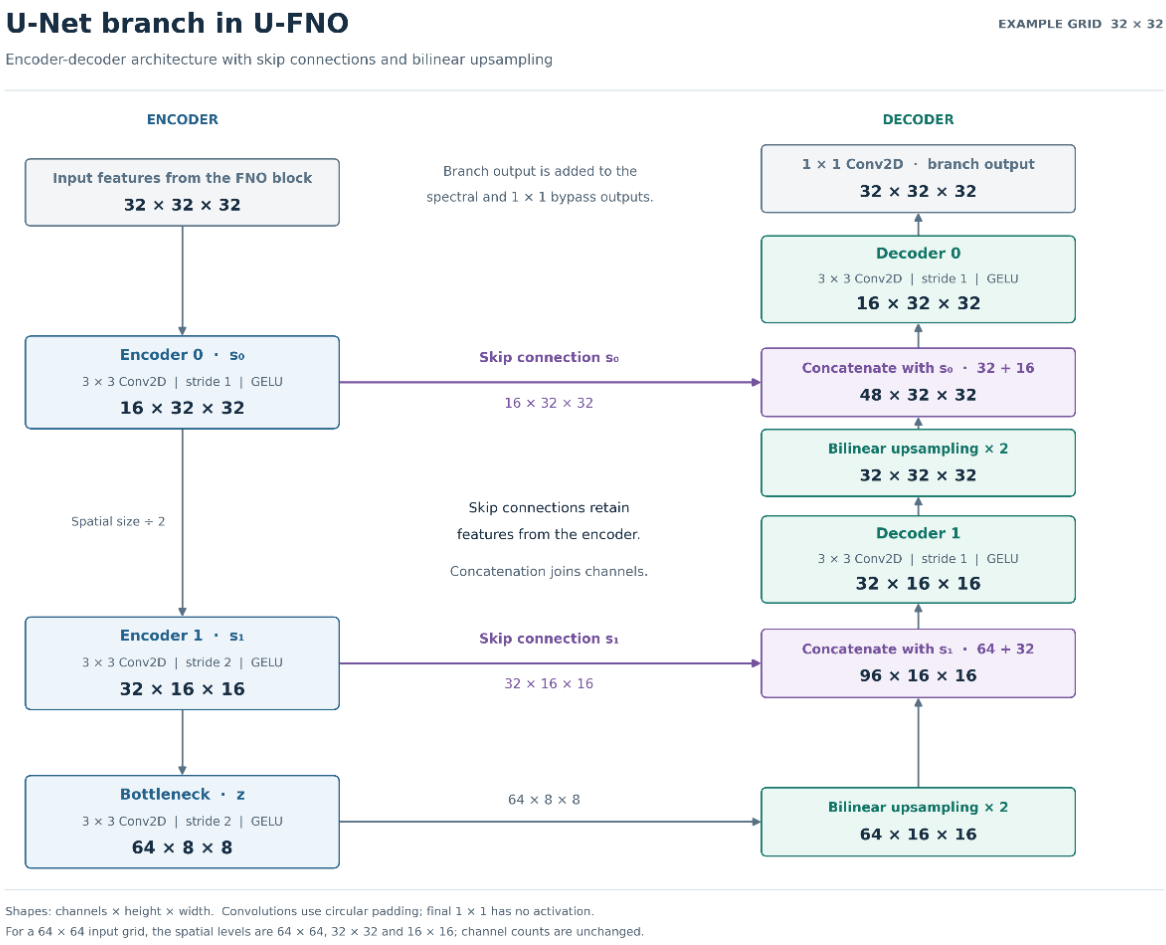
Based on this paper (https://arxiv.org/pdf/2109.03697)

Structure of each U-FNO block is simply:

GELU(Spectral conv branch + Bypass conv branch + U-Net branch)



The structure of the U-Net branch is as follows:



Upsampling can be implemented by simply repeating the feature in both directions, or by bilinear interpolation.

All conv2D layers use a 3x3 kernel.

Note that dropping from 32 to 16 for the first input feature after Encode 0 is not a requirement. 32 to 32 would make more sense, but that would make the output of the bottleneck be 128x8x8 which is pretty hefty. In the paper above they keep this number constant I think (32 to 32 to 32 after the bottleneck) (they have github link for their implementation). However, this number should actually increase as that is the original design/property of a U-NET.

I hope this quick explanation make sense to you. If not, we can discuss this when we next meet.

Implementation of the U-NET branch is shown below. I'm still testing out with different configurations and will update you with further details later on. But this is the baseline version.

```
class UNet2d(eqx.Module):
    enc0: eqx.nn.Conv2d
    enc1: eqx.nn.Conv2d
    bottleneck: eqx.nn.Conv2d
    dec1: eqx.nn.Conv2d
    dec0: eqx.nn.Conv2d
    output_conv: eqx.nn.Conv2d

    def __init__(self, width, base_width=16, *, key):
        keys = jax.random.split(key, 6)
        c = base_width
        self.enc0 = eqx.nn.Conv2d(width, c, 3, padding=1,
                                  padding_mode='CIRCULAR', key=keys[0])
        self.enc1 = eqx.nn.Conv2d(c, 2*c, 3, stride=2, padding=1,
                                  padding_mode='CIRCULAR', key=keys[1])
        self.bottleneck = eqx.nn.Conv2d(2*c, 4*c, 3, stride=2, padding=1,
                                        padding_mode='CIRCULAR', key=keys[2])
        self.dec1 = eqx.nn.Conv2d(4*c + 2*c, 2*c, 3, padding=1,
                                  padding_mode='CIRCULAR', key=keys[3])
        self.dec0 = eqx.nn.Conv2d(2*c + c, c, 3, padding=1,
                                  padding_mode='CIRCULAR', key=keys[4])
        self.output_conv = eqx.nn.Conv2d(c, width, 1, key=keys[5])


    def __call__(self, x):
        # Grid sizes must be divisible by four.
        s0 = jax.nn.gelu(self.enc0(x))           # (16,n,n): first skip
        s1 = jax.nn.gelu(self.enc1(s0))          # (32,n/2,n/2): second skip
        z = jax.nn.gelu(self.bottleneck(s1))     # (64,n/4,n/4)

        z = jnp.repeat(jnp.repeat(z, 2, axis=-2), 2, axis=-1)
        z = jnp.concatenate([z, s1], axis=0)     # (96,n/2,n/2)
        z = jax.nn.gelu(self.dec1(z))            # (32,n/2,n/2)

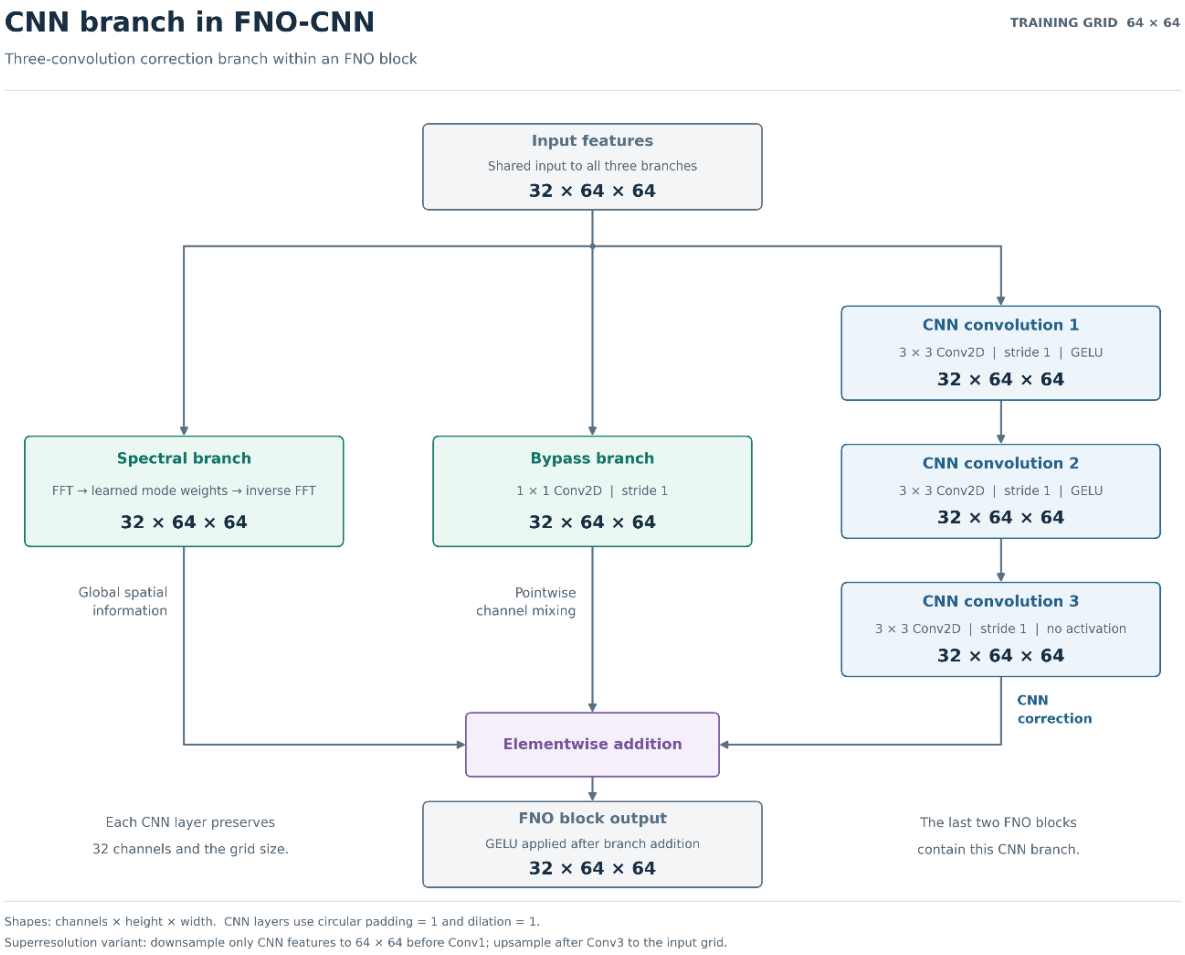
        z = jnp.repeat(jnp.repeat(z, 2, axis=-2), 2, axis=-1)
        z = jnp.concatenate([z, s0], axis=0)     # (48,n,n)
        z = jax.nn.gelu(self.dec0(z))            # (16,n,n)
        return self.output_conv(z)               # (32,n,n): correction
```

2.3 FNO-CNN model

This is simply the FNO model with an additional CNN branch.

GELU(Spectral conv branch + Bypass conv branch + CNN branch)

The CNN branch is simply 3 conv2D layers with 3x3 kernel:



Implementation is as follows:

```python
class FNOBlock2d(eqx.Module):
    spectral_conv: SpectralConv2d
    bypass_conv: eqx.nn.Conv2d
    cnn_conv1: Optional[eqx.nn.Conv2d]
    cnn_conv2: Optional[eqx.nn.Conv2d]
    cnn_conv3: Optional[eqx.nn.Conv2d]


    def __init__(self, width, modes, *, key, use_cnn=False):
        k1, k2 = jax.random.split(key)
        self.spectral_conv = SpectralConv2d(width, modes, key=k1)
        self.bypass_conv = eqx.nn.Conv2d(width, width, kernel_size=1, key=k2)

        self.cnn_conv1 = None
        self.cnn_conv2 = None
        self.cnn_conv3 = None
        if use_cnn:
            # Additional keys preserve the baseline spectral/bypass initialization.
            k3, k4, k5 = jax.random.split(jax.random.fold_in(key, 2), num=3)
            self.cnn_conv1 = eqx.nn.Conv2d(width, width, kernel_size=3, padding=1, padding_mode='CIRCULAR', key=k3)
            self.cnn_conv2 = eqx.nn.Conv2d(width, width, kernel_size=3, padding=1, padding_mode='CIRCULAR', key=k4)
            self.cnn_conv3 = eqx.nn.Conv2d(width, width, kernel_size=3, padding=1, padding_mode='CIRCULAR', key=k5)


    def __call__(self, x):
        y = self.spectral_conv(x) + self.bypass_conv(x)

        if self.cnn_conv1 is not None:
            cnn1 = jax.nn.gelu(self.cnn_conv1(x))
            cnn2 = jax.nn.gelu(self.cnn_conv2(cnn1))
            y = y + self.cnn_conv3(cnn2)

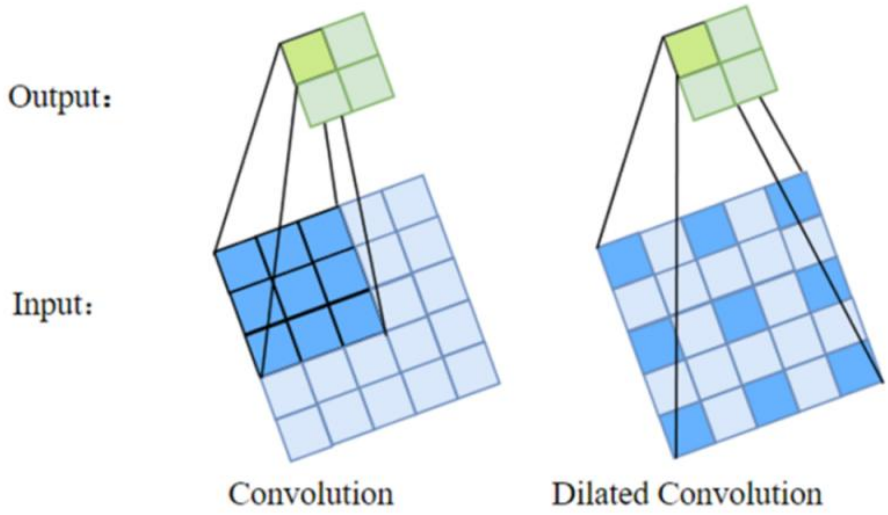
        return jax.nn.gelu(y)
```

3. Zero shot super resolution (zssr)

Two methods are applied for ZSSR for U-FNO and FNO-CNN are:

3.1 Dilation

Dilation is built into Conv2D in equinox already, so implementation is very simple. Basically, this setting spread the individual kernel values further apart.



Implementation is simply as below (notice the dilation argument in the eqx.nn.Conv2d function call):

```python
class FNOBlock2d(eqx.Module):
    spectral_conv: SpectralConv2d
    bypass_conv: eqx.nn.Conv2d
    cnn_conv1: Optional[eqx.nn.Conv2d]
    cnn_conv2: Optional[eqx.nn.Conv2d]
    cnn_conv3: Optional[eqx.nn.Conv2d]


    def __init__(self, width, modes, *, key, use_cnn=False, dilation=1):
        k1, k2 = jax.random.split(key)
        self.spectral_conv = SpectralConv2d(width, modes, key=k1)
        self.bypass_conv = eqx.nn.Conv2d(width, width, kernel_size=1, key=k2)

        self.cnn_conv1 = None
        self.cnn_conv2 = None
        self.cnn_conv3 = None
        if use_cnn:
            # Additional keys preserve the baseline spectral/bypass initialization.
            k3, k4, k5 = jax.random.split(jax.random.fold_in(key, 2), num=3)
            self.cnn_conv1 = eqx.nn.Conv2d(
                width, width, kernel_size=3, dilation=dilation,
                padding=dilation, padding_mode='CIRCULAR', key=k3)
            self.cnn_conv2 = eqx.nn.Conv2d(
                width, width, kernel_size=3, dilation=dilation,
                padding=dilation, padding_mode='CIRCULAR', key=k4)
            self.cnn_conv3 = eqx.nn.Conv2d(
                                width, width, kernel_size=3, dilation=dilation,
                                padding=dilation, padding_mode='CIRCULAR', key=k5)
```

3.2 Down-sampling the input for the CNN/U-NET branch

Using the scale_and_translate function already implemented in jax, we can simply down sample the larger input (from 256 to 64) to the resolution the CNN kernel was trained on, apply the kernel, up sample it again to 256, then add it into the other 2 branches as usual. Seem like a very simple approach but actually very effective.

```python
def resize_periodic(x, size):
    # Linear resampling on the periodic grid x_i = i * L / n.
    n_input = x.shape[-1]
    if n_input == size:
        return x

    scale = size / n_input
    padding = max(1, (n_input + size - 1) // size)
    padded = jnp.pad(x, ((0, 0), (padding, padding), (padding, padding)), mode='wrap')
    translation = 0.5 - scale * (padding + 0.5)
    # new input start: padding + 0.5
    # output start should be 0.5
    # therefore, translation = output_start - scale * new_input_start

    return jax.image.scale_and_translate(
        padded, shape=(x.shape[0], size, size), spatial_dims=(1, 2),
        scale=jnp.array([scale, scale]),
        translation=jnp.array([translation, translation]),
        method='linear', antialias=True)
    # spatial_dims resize columns 1 and 2 for xx and yy
```

This is bilinear interpolation. Anti-alias helps smooth sharp transitions and remove high-frequency artifacts for better interpolation (for down-sampling only. It doesn't do anything for up-sampling).

The function is applied to the network as follows:

```
def __call__(self, x, unet_resolution=None):
    y = self.spectral_conv(x) + self.bypass_conv(x)
    if self.unet is not None:
        if unet_resolution is None:
            correction = self.unet(x)
        else:
            unet_input = resize_periodic(x, unet_resolution)
            correction = self.unet(unet_input)
            correction = resize_periodic(correction, x.shape[-1])
        y = y + correction
    return jax.nn.gelu(y)
```

4. Results

The general training results are shown here:

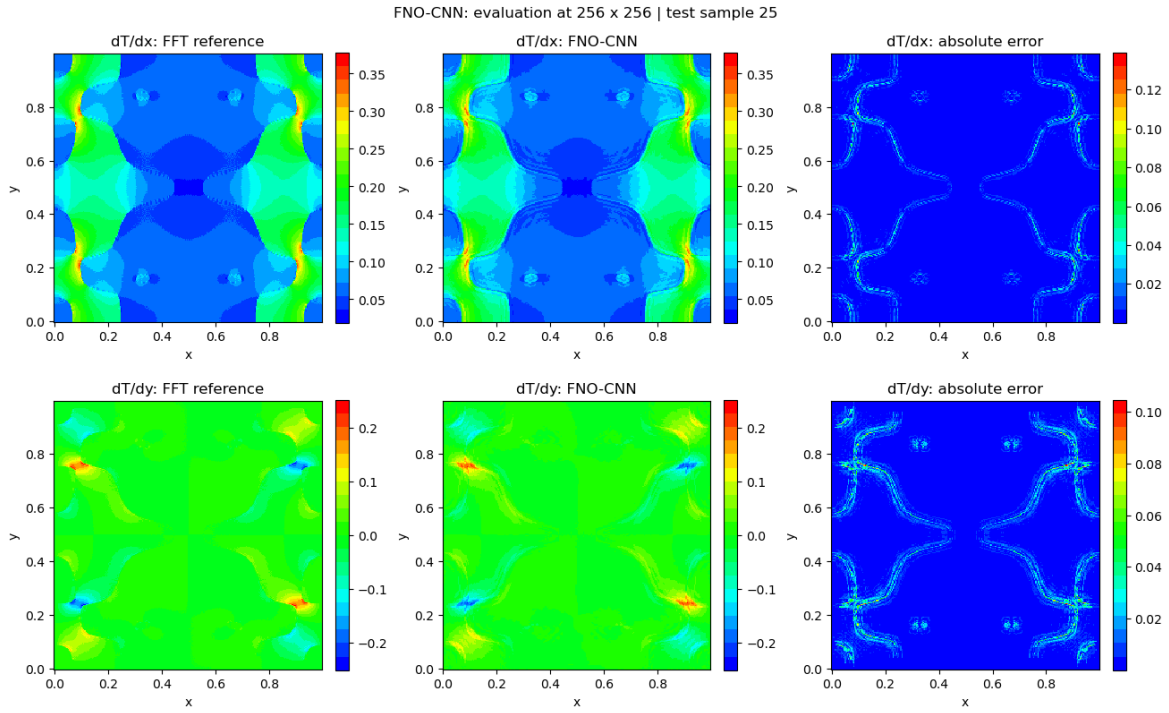
	Regular FNO	FNO with width = 33	U-FNO	FNO-CNN
Number of parameters	**3,149,090**	3,348,974	3,274,914	3,204,578
Total training time (minutes)	**371.0**	503.8	1343.0	875.1
Final loss	9.9875E-07	8.9477E-07	9.7017E-08	**8.9862E-08**
Mean L2 error	1.1864%	1.1010%	0.4185%	**0.3929%**

The results for ZSSR without and with the two remedies are:

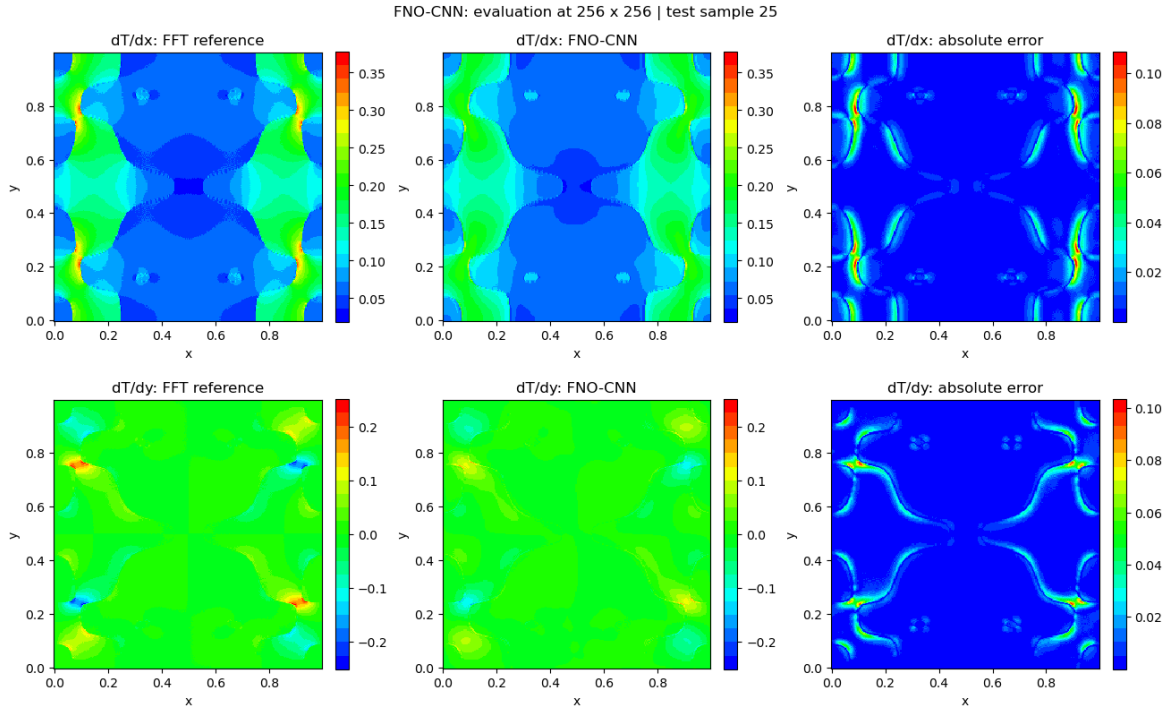
Mean L2 error	64x64	256x256
Regular FNO	1.1864%	1.4931%
U-FNO	0.4185%	3.5545%
U-FNO with dilation		5.1448%
U-FNO with down-sampled input		**1.8111%**
FNO-CNN	0.3930%	5.1513%
FNO-CNN with dilation		**1.4895%**
FNO-CNN with down-sampled input		2.2256%

5. Interesting graphs

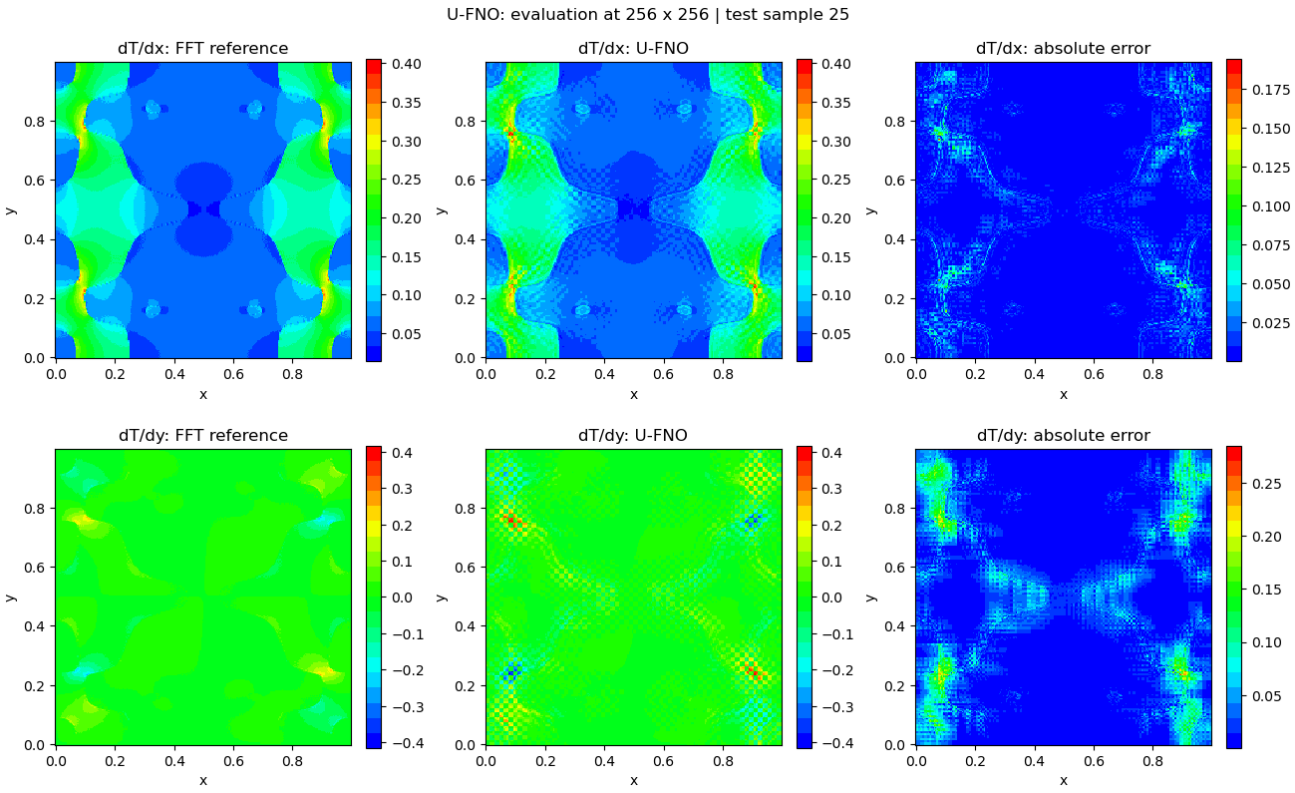
The rest of the graphs can be viewed in the jupyter notebook files. As the report is already so long, I will only include some of the most interesting graphs.



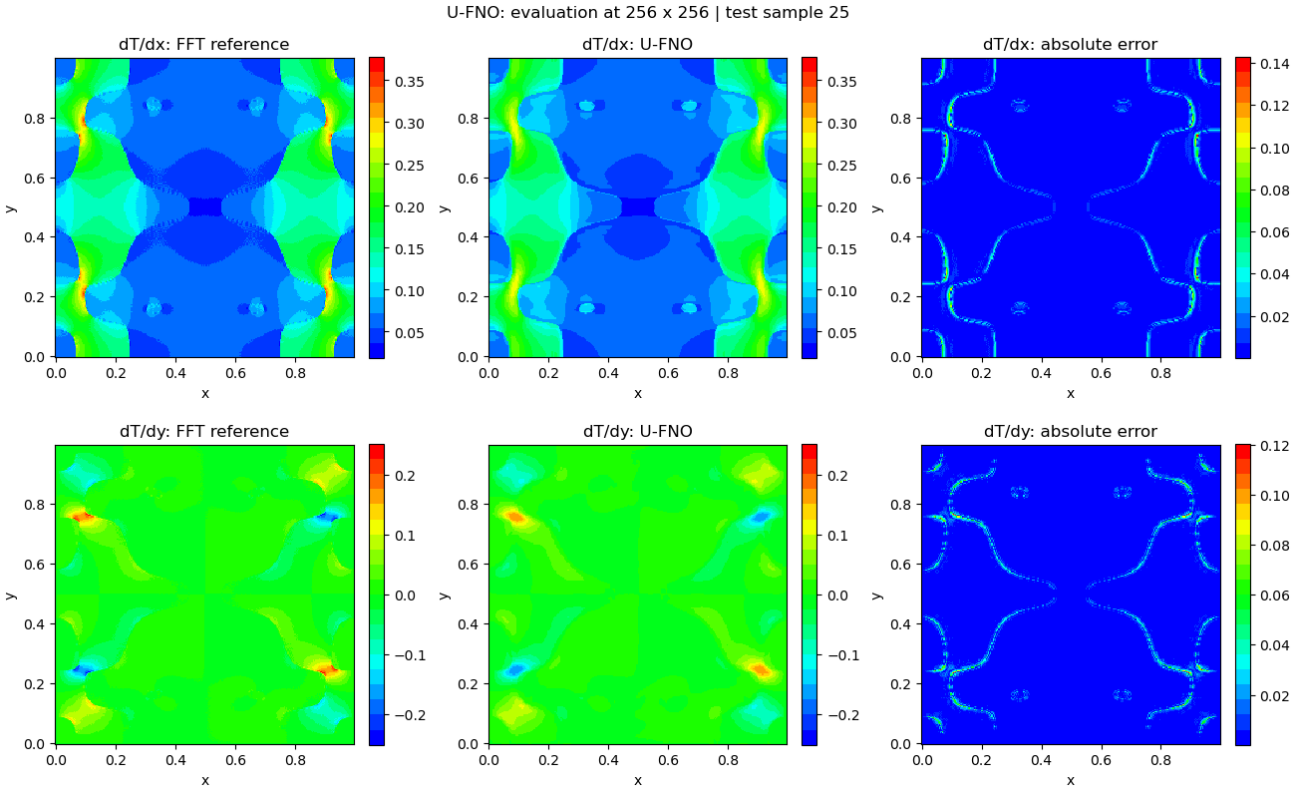
This is: ZSSR of FNO-CNN with dilation. Very prominent artifact despite having a mean L2 error of 1.48% (lowest out of all methods). Maybe this is not the Gibbs phenomenon but the artifact of using dilation.



This is regular FNO-CNN and you don’t see the artifact at all.



This is U-FNO with dilation. The artifact is even more obvious with very prominent checker-board pattern.



This is U-FNO with down-sampled input for the CNN branch.